

Sección 5: Orquestar la coordinación de múltiples agentes (15-20%)

⚠ **Sección de ALTO PESO en el examen.** Esta sección representa entre el 15% y el 20% de las preguntas. Dominar la orquestación multi-agente es fundamental para aprobar.

5.1 Funcionamiento y administración de flujos de trabajo de varios agentes

Conceptos clave

Patrones de orquestación multi-agente

Existen cuatro patrones fundamentales para coordinar múltiples agentes. Es esencial conocer cuándo aplicar cada uno:

Patrón	Descripción	Cuándo usarlo	Ejemplo en GitHub
Fan-out (Paralelo)	Múltiples agentes trabajan simultáneamente en tareas independientes	Tareas sin dependencias entre sí	Varios Copilot coding agents trabajando en issues diferentes, cada uno en su propia rama
Pipeline (Secuencial)	Un agente pasa su resultado al siguiente en cadena	Tareas con dependencias lineales	Agente 1 genera código → Agente 2 escribe tests → Agente 3 documenta
Orquestador (Central)	Un agente central delega y coordina el trabajo de otros agentes	Tareas complejas que requieren coordinación centralizada	Un agente principal descompone un issue grande en sub-issues y asigna cada uno a un agente diferente
Peer Review (Revisión entre pares)	Los agentes revisan el trabajo de otros agentes	Validación cruzada y calidad	Un agente genera código y otro agente revisa el PR resultante

Principios de coordinación

- **Descomposición de tareas:** Dividir el trabajo en unidades atómicas e independientes cuando sea posible.
- **Definición clara de interfaces:** Cada agente debe tener entradas y salidas bien definidas.
- **Minimización de dependencias:** Reducir las interacciones entre agentes para evitar cuellos de botella.
- **Idempotencia:** Las operaciones de los agentes deben poder repetirse sin efectos secundarios no deseados.

Configuración del aislamiento para ejecución en paralelo

El aislamiento es crítico cuando múltiples agentes trabajan simultáneamente. En el ecosistema GitHub, el aislamiento se logra mediante:

1. Ramas separadas (branches):

- Cada agente trabaja en su propia rama feature.
- Convención de nomenclatura: `copilot/agent-<id>/<task>` o `copilot/fix-<issue-number>`.
- Las ramas proporcionan aislamiento a nivel de código fuente.

2. Pull Requests independientes:

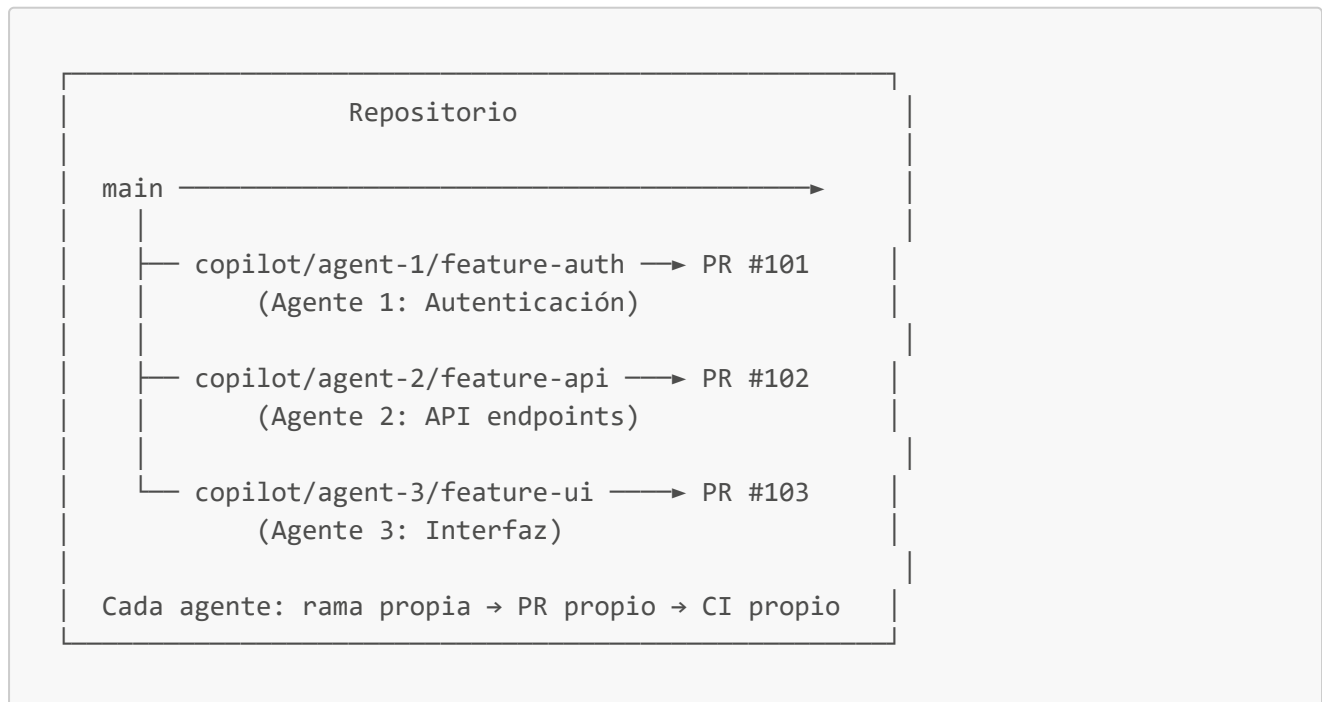
- Cada agente crea su propio PR.
- Los PRs actúan como unidades de trabajo aisladas.
- Permiten revisión y aprobación independiente.

3. Ejecuciones de CI separadas:

- Cada PR dispara su propia ejecución de GitHub Actions.
- Los tests se ejecutan en entornos aislados (runners independientes).
- Los resultados de CI son específicos de cada agente/PR.

4. Contextos de ejecución independientes:

- Cada sesión de Copilot coding agent tiene su propio entorno.
- No comparten estado en memoria ni sistema de archivos durante la ejecución.
- Los agentes personalizados en VS Code (archivos `agent.md`) operan en contextos separados.



Detección y resolución de conflictos entre agentes

Los conflictos multi-agente se clasifican en tres categorías:

1. Cambios de código superpuestos (Overlapping Changes)

- **Detección:** Conflictos de merge al intentar integrar ramas. GitHub muestra el estado "This branch has conflicts that must be resolved".
- **Señales tempranas:** Dos PRs modifican los mismos archivos (visible en la pestaña "Files changed" de cada PR).
- **Herramientas:** `git diff`, comparación de PRs, GitHub code owners alertando sobre archivos compartidos.

2. Esfuerzo duplicado (Duplicate Effort)

- **Detección:** Dos agentes producen funcionalidad equivalente o muy similar.
- **Señales:** Issues que se solapan en alcance, PRs con descripciones o cambios similares.
- **Prevención:** Descomposición clara de tareas, uso de GitHub Issues como fuente única de verdad para la asignación de trabajo.

3. Salidas opuestas (Contradictory Outputs)

- **Detección:** Un agente implementa una solución que contradice la de otro (por ejemplo, uno usa REST y otro GraphQL para el mismo endpoint).
- **Señales:** Tests que pasan individualmente pero fallan al integrar, comportamiento inconsistente en la aplicación.
- **Prevención:** Definir estándares y restricciones en las instrucciones del agente antes de iniciar el trabajo.

Estrategias de resolución:

Estrategia	Descripción	Cuándo aplicar
Rebase	Rebase de la rama del agente sobre la rama principal actualizada	Conflictos menores, cambios no contradictorios
Merge manual	Resolución manual de conflictos de merge	Cambios superpuestos que requieren decisión humana
Arbitraje humano	Un revisor humano decide qué cambios prevalecen	Salidas opuestas o decisiones arquitectónicas
Re-ejecución del agente	Volver a ejecutar el agente con contexto actualizado	Cuando el conflicto se debe a estado desactualizado

Aplicación práctica en GitHub

Escenario: Desarrollo paralelo de una feature compleja

1. **Descomposición:** Crear un issue principal y sub-issues en GitHub Issues.

```
Issue #50: Implementar sistema de notificaciones
├─ Issue #51: Backend - Servicio de notificaciones (→ Agente 1)
├─ Issue #52: API - Endpoints REST (→ Agente 2)
└─ Issue #53: Frontend - Componentes UI (→ Agente 3)
```

2. **Asignación:** Asignar cada sub-issue a Copilot (usando "Assign to Copilot" en cada issue).
3. **Monitoreo:** Revisar los PRs generados, verificar que no haya conflictos entre ramas.
4. **Integración:** Merge secuencial de los PRs, resolviendo conflictos si existen.

Configuración de agentes personalizados en VS Code:

- Los archivos `.github/agents/*.md` o `agent.md` definen agentes con instrucciones específicas.
- Cada agente puede tener herramientas (tools) y restricciones diferentes.
- El aislamiento de contexto se logra definiendo el alcance de cada agente.

Puntos clave para el examen

- ☒ Conocer los **cuatro patrones de orquestación** y saber cuándo aplicar cada uno.
- ☒ El aislamiento en GitHub se logra mediante **ramas separadas, PRs independientes y ejecuciones de CI aisladas**.
- ☒ Los tres tipos de conflictos: **cambios superpuestos, esfuerzo duplicado y salidas opuestas**.
- ☒ Las estrategias de resolución: **rebase, merge manual, arbitraje humano y re-ejecución**.
- ☒ **GitHub Issues como mecanismo de descomposición de tareas** para trabajo multi-agente.
- ☒ El patrón **fan-out** es el más natural para múltiples sesiones de Copilot coding agent en paralelo.
- ☒ El patrón **pipeline** requiere definir claramente las interfaces entre agentes (entrada/salida de cada paso).
- ☒ La **prevención de conflictos** es preferible a la resolución: buena descomposición de tareas minimiza conflictos.

5.2 Configurar observabilidad para comportamiento multi-agente

Conceptos clave

¿Qué es la observabilidad multi-agente?

La observabilidad en sistemas multi-agente es la capacidad de entender, rastrear y auditar el comportamiento y las decisiones de cada agente y sus interacciones. Se basa en tres pilares:

1. **Trazabilidad (Tracing):** Seguir el flujo de trabajo a través de múltiples agentes.
2. **Registro (Logging):** Documentar las acciones y decisiones de cada agente.
3. **Artefactos (Artifacts):** Generar evidencia tangible del trabajo realizado.

Generación de artefactos para revisión y auditoría

En el ecosistema GitHub, los artefactos de observabilidad multi-agente incluyen:

1. Pull Request Descriptions como documentos de transferencia (handoff docs)

- El cuerpo del PR documenta qué hizo el agente, por qué y cómo.
- Incluyen el contexto del issue original, las decisiones tomadas y los cambios realizados.
- Actúan como "contrato" entre el agente y el revisor humano.
- Formato recomendado:

```
## Contexto
Resuelve #51 - Servicio de notificaciones

## Decisiones tomadas
- Se eligió patrón Observer para desacoplar emisores de receptores
- Se usó Redis como broker por requisitos de rendimiento

## Cambios realizados
- src/services/notification.ts: Nuevo servicio
- src/models/notification.ts: Modelo de datos
- tests/notification.test.ts: Tests unitarios (12/12 passed)

## Dependencias
- Requiere merge de PR #102 (API endpoints) antes de integración final
```

2. GitHub Actions Artifacts

- Los workflows de CI/CD pueden generar y almacenar artefactos (logs, reportes, capturas).
- Útiles para auditoría post-ejecución.
- Se configuran con la acción `actions/upload-artifact`.
- Ejemplo de configuración:

```
- name: Upload agent execution report
  uses: actions/upload-artifact@v4
  with:
    name: agent-execution-report
    path: reports/agent-output.json
```

3. Commit messages como logs de decisiones

- Cada commit documenta una decisión o acción del agente.
- Mensajes descriptivos que explican el "por qué" además del "qué".
- El historial de commits proporciona una línea temporal de las acciones del agente.
- Convención: `[Agent-ID] tipo: descripción` (ejemplo: `[copilot] feat: add notification service with Observer pattern`).

4. Issue comments y timeline

- Los agentes pueden comentar en issues para documentar progreso.
- La timeline del issue muestra la secuencia de eventos y decisiones.

- Los labels y las actualizaciones de estado documentan el ciclo de vida.

Documentar decisiones, transferencias y resultados entre agentes

Las **transferencias (handoffs)** entre agentes deben documentarse explícitamente:

Elemento	Dónde documentar	Qué incluir
Decisiones clave	Commit messages, PR description	Alternativas consideradas, razón de la elección
Transferencias	Issue comments, PR cross-references	Qué se entrega, qué falta, dependencias
Resultados	PR description, CI artifacts	Estado final, métricas, tests pasados/fallidos
Contexto compartido	Issue body, linked PRs	Información que el siguiente agente necesita

Vinculación entre artefactos:

- Usar referencias cruzadas entre issues y PRs (**Closes #51, Related to #52**).
- Mantener un issue padre con links a todos los sub-issues y PRs.
- Usar GitHub Projects para visualizar el estado general del flujo multi-agente.

Análisis post-hoc del comportamiento multi-agente

El análisis post-hoc permite evaluar la eficacia del flujo multi-agente después de su ejecución:

1. Revisión de la línea temporal

- Reconstruir la secuencia de eventos usando el timeline de issues y PRs.
- Identificar cuellos de botella (agentes que tardaron mucho más que otros).
- Detectar patrones de conflicto recurrentes.

2. Métricas clave

- **Tiempo total del flujo:** Desde la creación del issue hasta el merge del último PR.
- **Tiempo por agente:** Duración de cada sesión de agente.
- **Tasa de conflictos:** Porcentaje de PRs que tuvieron conflictos de merge.
- **Tasa de re-ejecución:** Cuántas veces hubo que re-ejecutar un agente.
- **Tasa de intervención humana:** Frecuencia con que un humano tuvo que intervenir.

3. Herramientas de análisis

- `git log --graph --oneline --all`: Visualizar la estructura de ramas y merges.
- GitHub Insights: Métricas de contribución y actividad.
- GitHub Actions run history: Historial de ejecuciones de CI/CD.
- Audit logs de la organización: Eventos de seguridad y administración.

4. Revisión de calidad

- Comparar la salida de cada agente con las especificaciones del issue.
- Evaluar la consistencia entre las salidas de diferentes agentes.
- Verificar que todas las decisiones documentadas son trazables hasta un requisito.

Aplicación práctica en GitHub

Configurar un flujo multi-agente observable:

1. Plantilla de issue con checklist de observabilidad:

```
## Tarea para Agente
[Descripción de la tarea]

## Requisitos de observabilidad
- [ ] PR description completa con decisiones y contexto
- [ ] Commits atómicos con mensajes descriptivos
- [ ] Tests incluidos y pasando
- [ ] Referencias cruzadas a issues relacionados
```

2. Workflow de CI que genera artefactos de auditoría:

```
name: Agent Audit Trail
on: pull_request
jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Generate change report
        run: |
          echo "## Change Report" > report.md
          echo "PR: #${{ github.event.pull_request.number }}" >>
report.md
          echo "Files changed:" >> report.md
          git diff --name-only origin/main >> report.md
      - uses: actions/upload-artifact@v4
        with:
          name: audit-report
          path: report.md
```

3. Dashboard en GitHub Projects:

- Columnas: Asignado → En Progreso → PR Creado → En Revisión → Completado.
- Cada tarjeta vinculada a un issue y su PR correspondiente.

Puntos clave para el examen

- ☒ Los **tres pilares de observabilidad**: trazabilidad, registro y artefactos.
 - ☒ **PR descriptions** son el principal mecanismo de documentación de transferencias entre agentes.
 - ☒ **Commit messages** actúan como logs de decisiones del agente.
 - ☒ **GitHub Actions artifacts** permiten almacenar evidencia para auditoría.
 - ☒ Las **referencias cruzadas** entre issues y PRs (**Closes #X**, **Related to #Y**) son fundamentales para la trazabilidad.
 - ☒ El análisis post-hoc requiere métricas: **tiempo, tasa de conflictos, tasa de re-ejecución, tasa de intervención humana**.
 - ☒ **GitHub Projects** proporciona visibilidad del estado general del flujo multi-agente.
 - ☒ La documentación de **decisiones clave** debe incluir alternativas consideradas y la razón de la elección final.
 - ☒ Los **audit logs** de la organización son relevantes para seguridad y cumplimiento.
-

5.3 Detección y respuesta a errores multi-agente y comportamiento degradado

Conceptos clave

Identificar ejecuciones fallidas, parciales o atascadas

Las ejecuciones de agentes pueden fallar de tres maneras:

1. Ejecución fallida (Failed)

- El agente encuentra un error que no puede resolver y se detiene.
- **Señales en GitHub:**
 - La sesión de Copilot coding agent muestra estado de error.
 - No se crea PR o se crea un PR vacío/incompleto.
 - GitHub Actions checks fallan (estado "failure").
 - El agente comenta en el issue indicando que no pudo completar la tarea.
- **Causas comunes:**
 - Instrucciones ambiguas o contradictorias.
 - Falta de permisos o acceso a recursos necesarios.
 - Errores de compilación o tests que el agente no puede resolver.
 - Límites de contexto o tiempo excedidos.

2. Ejecución parcial (Partial)

- El agente completa parte del trabajo pero no todo.
- **Señales:**
 - PR creado pero con TODO/FIXME pendientes en el código.
 - Tests parciales: algunos pasan, otros fallan o faltan.
 - La descripción del PR indica trabajo incompleto.
 - Issue no se cierra automáticamente (falta el **Closes #X** o no cumple todos los criterios).
- **Riesgos:**
 - Dar por completado un trabajo que está a medias.
 - Construir sobre una base incompleta (otros agentes dependen de este resultado).

3. Ejecución atascada (Stuck)

- El agente entra en un bucle o deja de hacer progreso sin terminar.
- **Señales:**
 - Sesión de agente que lleva mucho tiempo sin nuevos commits.
 - Commits repetitivos que no avanzan (revert + retry loops).
 - CI ejecutándose repetidamente sin cambios significativos.
 - Ausencia de actividad en el PR o issue por período prolongado.
- **Causas comunes:**
 - Bucles de corrección infinitos (el agente intenta arreglar algo y rompe otra cosa).
 - Dependencias circulares entre tareas.
 - Esperando un recurso o aprobación que nunca llega.

Responder ante comportamiento o coordinación degradada

El comportamiento degradado ocurre cuando los agentes funcionan pero su coordinación es subóptima:

Tipos de degradación:

1. **Desincronización temporal:** Un agente avanza mucho más rápido que otro, generando dependencias no resueltas.
2. **Deriva de contexto:** Los agentes pierden coherencia con el objetivo original conforme avanzan.
3. **Conflictos en cascada:** Un conflicto no resuelto afecta a múltiples agentes downstream.
4. **Redundancia no detectada:** Múltiples agentes implementan la misma funcionalidad sin saberlo.

Estrategias de respuesta:

Situación	Respuesta inmediata	Acción correctiva
Agente atascado	Cancelar la sesión del agente	Re-ejecutar con instrucciones más claras o contexto adicional
Ejecución parcial	Revisar el PR parcial	Crear un nuevo issue para el trabajo restante y asignar a un agente
Conflictos en cascada	Pausar agentes downstream	Resolver el conflicto raíz y luego reanudar
Desincronización	Identificar el agente bloqueante	Priorizar el agente lento o reasignar su tarea
Deriva de contexto	Revisar las instrucciones del agente	Actualizar las instrucciones y re-ejecutar si es necesario

Patrones de recuperación: reversión e intervención humana

1. Reversión (Rollback)

- **A nivel de rama:** Eliminar la rama del agente y crear una nueva desde **main**.

```
git branch -D copilot/agent-1/feature-broken
git checkout -b copilot/agent-1/feature-retry main
```

- **A nivel de commit:** Revertir commits específicos del agente.

```
git revert <commit-sha>
```

- **A nivel de PR:** Cerrar el PR sin merge y crear uno nuevo.
- **A nivel de deploy:** Si cambios ya se mergearon, usar el botón "Revert" del PR en GitHub para crear un PR de reversión automático.

2. Intervención humana (Human-in-the-Loop)

- **Revisión obligatoria de PRs:** Configurar branch protection rules que requieran aprobación humana antes del merge.

```
# .github/settings.yml (si se usa probot/settings)
branches:
  - name: main
    protection:
      required_pull_request_reviews:
        required_approving_review_count: 1
```

- **Puntos de control (checkpoints):** Definir momentos en el flujo donde un humano debe validar antes de continuar.
- **Escalación:** Cuando un agente falla repetidamente, escalar a un desarrollador humano.
- **Aprobación de GitHub Actions:** Usar `environment` con revisores requeridos para gates de aprobación manual.

```
jobs:
  deploy:
    environment:
      name: production
      # Requiere aprobación manual antes de ejecutar
```

3. Patrón Circuit Breaker

- Si un agente falla N veces consecutivas, dejar de re-ejecutarlo automáticamente.
- Notificar al equipo y esperar intervención humana.
- Evita consumo innecesario de recursos en bucles de fallo.

4. Patrón Compensación

- Si un agente produjo un resultado incorrecto que otros agentes ya usaron como base:
 1. Identificar todos los agentes afectados.
 2. Revertir los cambios del agente original.
 3. Re-ejecutar los agentes afectados con el contexto correcto.

Aplicación práctica en GitHub

Monitoreo de sesiones de agente:

1. **Verificar estado del agente:** Revisar la sesión de Copilot coding agent en la interfaz de GitHub.
2. **Revisar CI status checks:** Dashboard de GitHub Actions para ver el estado de cada PR.
3. **Configurar notificaciones:** Usar GitHub notifications o webhooks para alertas de fallos.

Workflow de recuperación ante fallo de agente:

- ```
1. Detectar fallo (CI rojo, PR sin actividad, comentario de error)
 |
2. Evaluar severidad
 ├── Menor: Re-ejecutar el agente con mejores instrucciones
 ├── Moderado: Intervención humana para resolver manualmente
 └── Crítico: Revertir todos los cambios y replantear la estrategia
 |
3. Documentar el fallo y la solución en el issue
 |
4. Actualizar instrucciones del agente para prevenir recurrencia
```

### Configurar alertas automáticas con GitHub Actions:

```
name: Agent Health Check
on:
 schedule:
 - cron: '0 */6 * * *' # Cada 6 horas
jobs:
 check-stale-prs:
 runs-on: ubuntu-latest
 steps:
 - name: Check for stale agent PRs
 uses: actions/github-script@v7
 with:
 script: |
 const prs = await github.rest.pulls.list({
 owner: context.repo.owner,
 repo: context.repo.repo,
 state: 'open',
 head: 'copilot/'
 });
 for (const pr of prs.data) {
 const hoursSinceUpdate =
 (Date.now() - new Date(pr.updated_at)) / 3600000;
```

```

 if (hoursSinceUpdate > 24) {
 console.log(`Stale agent PR: #${pr.number}`);
 // Crear notificación o issue
 }
 }
}

```

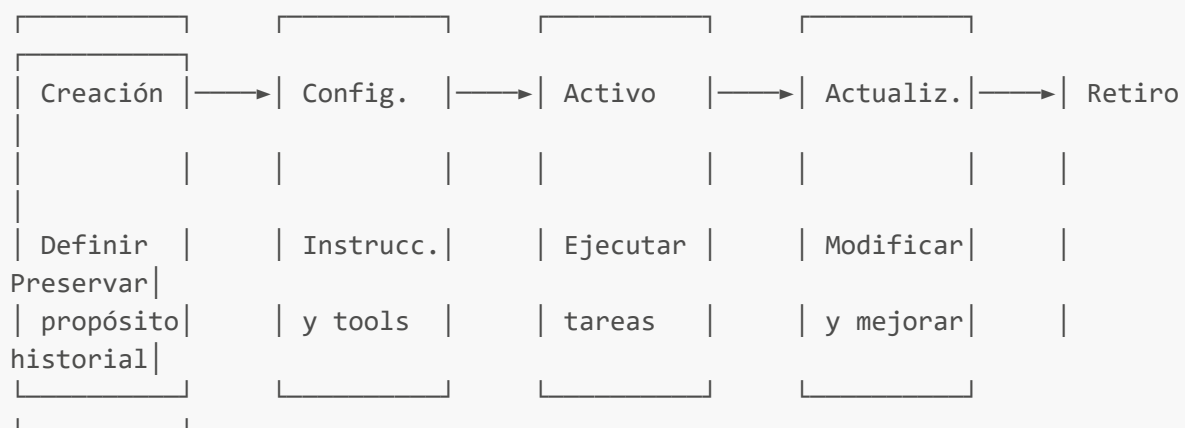
## Puntos clave para el examen

- ☒ Tres estados de fallo: **fallida, parcial y atascada** — saber identificar cada una.
- ☒ Las **señales de ejecución atascada**: falta de commits nuevos, bucles de revert+retry, CI repetitivo sin avance.
- ☒ **Reversión a nivel de PR** usando el botón "Revert" de GitHub crea automáticamente un PR de reversión.
- ☒ **Branch protection rules** son el mecanismo principal para forzar intervención humana.
- ☒ El patrón **Circuit Breaker** evita re-ejecuciones infinitas de agentes fallidos.
- ☒ Los **environments con revisores requeridos** en GitHub Actions permiten gates de aprobación manual.
- ☒ La **compensación** requiere identificar todos los agentes afectados por un fallo upstream.
- ☒ La **documentación del fallo** en el issue es parte esencial del proceso de recuperación.
- ☒ El comportamiento degradado incluye: **desincronización, deriva de contexto, conflictos en cascada y redundancia**.
- ☒ Siempre **actualizar las instrucciones del agente** después de resolver un fallo para prevenir recurrencia.

## 5.4 Administrar el ciclo de vida de los agentes dentro de flujos multi-agente

### Conceptos clave

#### Fases del ciclo de vida de un agente



#### Adición de agentes a flujos de trabajo existentes

### En GitHub (agentes en la nube - Copilot coding agent):

- Asignar Copilot a un nuevo issue mediante "Assign to Copilot" en la interfaz de GitHub.
- El agente se integra al flujo existente a través del sistema de issues y PRs.
- No requiere configuración adicional de infraestructura; utiliza la configuración del repositorio.

### En VS Code (agentes personalizados con archivos agent.md):

- Crear un archivo de definición del agente en `.github/agents/` o como `agent.md`.
- El archivo define: nombre, descripción, instrucciones, herramientas disponibles y restricciones.
- Estructura típica de un archivo de agente:

```

name: security-reviewer
description: Revisa código en busca de vulnerabilidades de seguridad
tools:
 - codeSearch
 - fileRead

Security Reviewer Agent

Instrucciones
Revisa el código propuesto en busca de vulnerabilidades comunes:
- Inyección SQL
- XSS
- Exposición de credenciales
- Dependencias con CVEs conocidos

Formato de salida
Genera un reporte con severidad (alta/media/baja) para cada hallazgo.
```

### Consideraciones al agregar agentes:

- **Compatibilidad:** Verificar que el nuevo agente no entre en conflicto con agentes existentes.
- **Alcance:** Definir claramente los límites del nuevo agente para evitar solapamiento.
- **Integración gradual:** Introducir el agente en tareas de bajo riesgo antes de usarlo en producción.
- **Documentación:** Actualizar la documentación del flujo para incluir al nuevo agente.

### Actualizar, reconfigurar o reemplazar agentes sin interrumpir flujos activos

#### Principios de actualización sin interrupción:

1. **Versiónado de configuración:** Mantener las instrucciones del agente bajo control de versiones (Git).
  - Los cambios en archivos `agent.md` o `.github/copilot-instructions.md` se rastrean en el historial de Git.
  - Permite rollback a versiones anteriores si la actualización causa problemas.

## 2. Actualización progresiva (Rolling Update):

- No cambiar la configuración de todos los agentes simultáneamente.
- Actualizar un agente, verificar su comportamiento, y luego actualizar los demás.
- En GitHub: modificar las instrucciones del agente en `.github/copilot-instructions.md` o en archivos `.github/agents/*.md`.

## 3. Estrategia Blue-Green para agentes:

- Crear una nueva versión del agente (nuevo archivo de configuración).
- Ejecutar ambas versiones en paralelo temporalmente.
- Validar que la nueva versión produce resultados correctos.
- Retirar la versión anterior una vez validada la nueva.

## 4. Reconfiguración en caliente:

- Modificar las instrucciones o herramientas disponibles del agente.
- Los cambios toman efecto en la siguiente ejecución del agente (no afectan sesiones en curso).
- Ejemplo: actualizar `.github/copilot-instructions.md` para cambiar el comportamiento de Copilot coding agent.

### Tipos de actualización:

| Tipo          | Qué cambia                       | Impacto                                      | Ejemplo                                                 |
|---------------|----------------------------------|----------------------------------------------|---------------------------------------------------------|
| Instrucciones | Prompt o guía del agente         | Bajo - cambia comportamiento, no capacidades | Actualizar <code>.github/copilot-instructions.md</code> |
| Herramientas  | Tools disponibles para el agente | Medio - amplía o restringe capacidades       | Agregar o quitar tools en <code>agent.md</code>         |
| Modelo        | Modelo de IA subyacente          | Alto - puede cambiar calidad y estilo        | Cambiar de un modelo a otro                             |
| Reemplazo     | Agente completo                  | Alto - nueva implementación                  | Crear nuevo <code>agent.md</code> , retirar el anterior |

### Retirar agentes conservando auditabilidad y continuidad

El retiro de un agente debe preservar la trazabilidad y no romper flujos activos:

#### 1. Preservación del historial:

- **Git history:** Todos los commits, PRs y cambios realizados por el agente permanecen en el historial de Git indefinidamente.
- **Issue history:** Los comentarios, labels y estados de issues asociados al agente se conservan.
- **PR history:** Los PRs cerrados/mergeados con su discussion permanecen accesibles.
- **Audit logs:** Los logs de auditoría de la organización registran las acciones del agente.

## 2. Proceso de retiro recomendado:

1. Verificar que no hay sesiones activas del agente
  - |
2. Documentar la razón del retiro en un commit o issue
  - |
3. Eliminar o archivar la configuración del agente
  - ├─ Eliminar archivo agent.md (el historial de Git lo preserva)
  - └─ O mover a un directorio de archivo: `.github/agents/retired/`
4. Actualizar flujos que dependían del agente
  - ├─ Reasignar tareas a otros agentes o humanos
  - └─ Actualizar documentación y workflows de CI/CD
5. Verificar que el flujo funciona sin el agente retirado

## 3. Continuidad del flujo:

- Antes de retirar, identificar todas las dependencias del agente en el flujo.
- Asignar las responsabilidades del agente retirado a otro agente o humano.
- Actualizar los workflows de GitHub Actions que referenciaban al agente.
- Comunicar el retiro al equipo para evitar confusión.

## 4. Conservación para auditoría:

- Los archivos eliminados de `.github/agents/` permanecen en el historial de Git.
- Se puede acceder a versiones anteriores con `git log -- .github/agents/retired-agent.md`.
- Los PRs y issues asociados mantienen las referencias cruzadas intactas.
- Consideración: si hay requisitos de cumplimiento, mantener un registro explícito del retiro.

## Aplicación práctica en GitHub

### Agregar un nuevo agente al flujo:

1. Crear el archivo de configuración: `.github/agents/new-agent.md`.
2. Definir instrucciones, herramientas y restricciones.
3. Probar en un issue de bajo riesgo.
4. Documentar en el README del proyecto.

### Actualizar un agente existente:

1. Modificar el archivo de configuración existente.
2. Hacer commit del cambio con mensaje descriptivo: `chore: update security-reviewer agent instructions to include OWASP Top 10 2024`.
3. Verificar en la siguiente ejecución del agente.

### Retirar un agente:

1. Crear issue documentando el retiro: "Retire legacy-formatter agent - replaced by new-formatter".

2. Verificar que no hay sesiones activas.
3. Eliminar el archivo de configuración via PR.
4. Actualizar workflows dependientes.
5. El historial de Git preserva toda la información para auditoría.

### Ejemplo de estructura de directorio para gestión de agentes:

```
.github/
├── copilot-instructions.md # Instrucciones globales de Copilot
├── agents/
│ ├── code-reviewer.md # Agente activo: revisión de código
│ ├── test-generator.md # Agente activo: generación de tests
│ ├── security-scanner.md # Agente activo: escaneo de seguridad
│ └── docs-writer.md # Agente activo: documentación
└── workflows/
 └── agent-health-check.yml # Monitoreo de salud de agentes
```

### Puntos clave para el examen

- ☒ **Agregar agente** = crear nuevo archivo de configuración en `.github/agents/` o asignar Copilot a un issue.
- ☒ **Actualizar agente** = modificar instrucciones/herramientas en el archivo de configuración; los cambios aplican en la **siguiente ejecución**, no afectan sesiones en curso.
- ☒ **Retirar agente** = eliminar configuración, pero el **historial de Git preserva toda la información** para auditoría.
- ☒ La **actualización progresiva (rolling update)** minimiza el riesgo de interrupción.
- ☒ Los archivos `agent.md` en VS Code y `.github/copilot-instructions.md` son los mecanismos principales de configuración.
- ☒ El retiro requiere: verificar no hay sesiones activas → documentar razón → eliminar config → reasignar tareas → verificar flujo.
- ☒ El **versionado de configuración en Git** permite rollback a versiones anteriores del agente.
- ☒ La **estrategia blue-green** permite validar un nuevo agente antes de retirar el anterior.
- ☒ Al retirar, siempre **reasignar las responsabilidades** del agente a otro agente o humano.
- ☒ Los **audit logs de la organización** complementan el historial de Git para cumplimiento normativo.

---

## Resumen rápido de la sección

| Subsección | Tema principal      | Concepto más importante                                                |
|------------|---------------------|------------------------------------------------------------------------|
| 5.1        | Flujos multi-agente | 4 patrones de orquestación + aislamiento por ramas/PRs/CI              |
| 5.2        | Observabilidad      | PR descriptions como handoff docs + commit messages como decision logs |

| Subsección | Tema principal         | Concepto más importante                                  |
|------------|------------------------|----------------------------------------------------------|
| 5.3        | Errores y recuperación | 3 estados de fallo + reversión + intervención humana     |
| 5.4        | Ciclo de vida          | Agregar/actualizar/retirar agentes preservando historial |

## Recursos

### Documentación oficial

- [GitHub Copilot coding agent](#) — Uso del agente de codificación de Copilot
- [GitHub Actions: Artifacts](#) — Almacenamiento y compartición de artefactos
- [Branch protection rules](#) — Reglas de protección de ramas
- [GitHub Issues: Sub-issues](#) — Seguimiento de trabajo con issues
- [Custom instructions for Copilot](#) — Instrucciones personalizadas

### Conceptos relacionados

- [Sección 3: Usar agentes de codificación de GitHub Copilot](#) — Fundamentos del agente individual
- [Sección 4: Personalización y extensión de agentes](#) — Configuración de agentes personalizados

### Términos clave para el examen

| Término                | Definición                                                                            |
|------------------------|---------------------------------------------------------------------------------------|
| <b>Fan-out</b>         | Patrón donde múltiples agentes trabajan en paralelo en tareas independientes          |
| <b>Pipeline</b>        | Patrón donde los agentes se ejecutan secuencialmente, pasando resultados al siguiente |
| <b>Orquestador</b>     | Agente central que delega y coordina el trabajo de otros agentes                      |
| <b>Peer Review</b>     | Patrón donde los agentes revisan el trabajo de otros agentes                          |
| <b>Handoff</b>         | Transferencia de trabajo, contexto y responsabilidad de un agente a otro              |
| <b>Circuit Breaker</b> | Patrón que detiene la re-ejecución automática tras N fallos consecutivos              |
| <b>Rolling Update</b>  | Actualización progresiva de agentes, uno a la vez, para minimizar riesgo              |
| <b>Blue-Green</b>      | Estrategia de ejecutar dos versiones del agente en paralelo durante la transición     |
| <b>Compensación</b>    | Revertir y re-ejecutar agentes afectados por un fallo upstream                        |
| <b>Aislamiento</b>     | Separación de contextos de ejecución mediante ramas, PRs y CI independientes          |